
COMPASS-KEM

设计规范与支撑文档

版本： 1.2

日期： 2026 年 6 月

作者：王更，汪哲东，夏雯雯，张宁岳，何文字，耿欣阳，段非，季炜丹，李继红

单位：上海交通大学密码与计算机安全实验室；上海智巡密码科技有限公司

状态：规范草案

目录

1 引言	1
2 算法描述	1
2.1 符号与记号	1
2.2 伪代码约定	3
2.2.1 数据类型	3
2.2.2 整数数组的算术运算	4
2.2.3 代数对象的表示	4
2.2.4 多项式和 NTT 表示的算术运算	5
2.2.5 矩阵和向量	5
2.2.6 矩阵和向量的算术运算	5
2.2.7 对数组应用算法：示例	6
2.3 基本操作	6
2.3.1 哈希	6
2.3.2 NTT 域表示	7
2.3.3 压缩与解压缩	8
2.4 COMPASS-PKE 算法	9
2.4.1 密钥生成 (KeyGen)	9
2.4.2 加密 (Encrypt)	10
2.4.3 解密 (Decrypt)	11
2.5 主算法	12
2.5.1 密钥生成	12
2.5.2 封装	12
2.5.3 解封装	13
3 设计原理	13
3.1 安全基础	13
3.2 参数选择	15
3.3 解密失败率分析	16
3.4 其他考虑因素	17
3.5 差异性声明	17
4 安全声明与分析	17
4.1 理论安全性	17
4.1.1 IND-CPA 安全性	18
4.1.2 经典随机预言机下的 IND-CCA 安全性	19

4.1.3	量子随机预言机下的 IND-CCA 安全性	19
4.2	具体安全性	19
4.2.1	原始攻击 (Primal Attack)	19
4.2.2	对偶攻击 (Dual Attack)	21
5	性能	22
5.1	性能分析	22
5.1.1	实现考量与权衡	22
5.1.2	跨安全级别的性能扩展	23
5.2	参考实现与优化实现	23
5.2.1	可移植 ANSI C 基线	23
5.2.2	AVX2 优化实现	24
6	特性声明	25

1. 引言

本规范定义了 COMPASS-KEM, 即 Compact Post-quantum Security Suite (紧凑后量子安全套件) 的一部分。它是一种基于模舍入学习 (Module Learning-with-Rounding, MLWR) 问题的后量子密钥封装机制 (KEM)。该算法旨在对抗大规模量子计算机带来的安全威胁, 支持四种安全类别, 分别对应 128 比特 (80 比特量子安全)、256 比特 (128 比特量子安全)、384 比特 (192 比特量子安全) 和 512 比特 (256 比特量子安全)。

算法的核心优势包括:

- **基于可证明安全的坚实基础:** 本方案的安全性基于素模数上的判定 MLWR 问题及其变体, 两者均具备从标准判定 MLWE 问题的归约, 从而继承了格问题的最坏情况复杂性。

早期 NIST 标准化进程中的 MLWR 方案如 SABER [DKRV18], 其安全性基于 2 的幂次模数上的 MLWR, 缺乏从标准格困难问题的安全归约。而素模数上的 MLWR 允许从 MLWE 归约, 因而具备更坚实的安全基础。此外, 素模数允许使用 NTT, 大幅提升了效率。

还需指出, SABER 原始论文 [DKRV18] 中的安全证明存在缺陷。我们通过引入 MLWR 问题的变体修复了这一问题, 该变体同样具有从 MLWE 的归约, 因此不影响方案的可证明安全性。

- **效率:** 与基于 MLWE 的方案 (如 ML-KEM) 相比, 噪声采样步骤被易于实现的压缩算法所替代。通过公钥压缩, 本方案在同等安全级别下享有更小的公钥规模 ($\leq 84\%$)。在 256 比特安全级别下, 密文规模也小于 ML-KEM。公钥压缩的代价是, 压缩后的公钥上的 NTT 操作必须在封装和解封装算法中执行, 而非在密钥生成算法中完成, 这会略微增加封装和解封装算法的时间开销 ($\leq 5\%$)。
- **无知识产权纠纷:** 通过避免使用 ML-KEM 中的误差项, 本方案规避了 ML-KEM 及其他基于 LWE 的方案所面临的知识产权问题。

2. 算法描述

2.1 符号与记号

算法的主要参数定义如下:

n 模多项式的次数。

q 整数模数。

ζ 表示模 q 下的 n 次本原单位根。

$\mathbb{B}$ 无符号 8 位整数（字节）的集合 $\{0, 1, \dots, 255\}$ 。

$\mathbb{Z}$ 整数集。

$\mathbb{Z}_m$ 模 m 的整数环（即集合 $\{0, 1, \dots, m-1\}$ ，配备模 m 加法和乘法运算）。

$\mathbb{Z}_m^n$ $\mathbb{Z}_m$ 上的 n 元组集合，配备 $\mathbb{Z}_m$ -模结构。作为数据类型，它是长度为 n 的数组，其元素属于 $\mathbb{Z}_m$ 。

R 环 $\mathbb{Z}[X]/(X^n + 1)$ 。

R_q 环 $\mathbb{Z}_q[X]/(X^n + 1)$ ，由形如 $f = f_0 + f_1X + \dots + f_{n-1}X^{n-1}$ 的多项式组成，其中对所有 j 有 $f_j \in \mathbb{Z}_q$ 。

T_q R_q 在数论变换下的像。其元素称为 R_q 中多项式的“NTT 表示”。

$\mathcal{D}_\eta(R_q)$ R_q 中具有小系数的多项式的一个特定分布，噪声由此采样。该分布由参数 $\eta \in \{2, 3, 4\}$ 参数化。

S^* 若 S 是一个集合，则表示 S 中元素的有限长度元组（或数组）的集合，包括空元组。

S^k 若 S 是一个集合，则表示 S 中元素的 k 元组（或长度为 k 的数组）的集合。

f_j 多项式 $f = f_0 + f_1X + \dots + f_{n-1}X^{n-1} \in R_q$ 中 X^j 的系数。

$\hat{f}$ T_q 中等于多项式 $f \in R_q$ 的 NTT 表示的元素。

$\mathbf{v}^T, \mathbf{A}^T$ 行向量或列向量 $\mathbf{v}$ 的转置。一般地，矩阵 $\mathbf{A}$ 的转置。

$\circ$ 表示系数在 R_q 或 T_q 中的线性代数组合（例如 $\mathbf{A} \circ \mathbf{v}$ ）。

$A \parallel B$ 两个数组或比特串 A 和 B 的拼接。

$B[i]$ 数组 B 中下标为 i 的元素。所有数组的下标从零开始。

$B[k:m]$ 数组 B 的子数组 $(B[k], B[k+1], \dots, B[m-1])$ 。

$|B|$ 若 B 是数字，表示 B 的绝对值。若 B 是数组，表示其长度。

$\lceil x \rceil$ x 的上取整（大于或等于 x 的最小整数）。

$\lfloor x \rfloor$ x 的下取整（小于或等于 x 的最大整数）。

$\lceil x \rceil$ x 四舍五入到最近整数。若 $x = y + 1/2$ ，其中 $y \in \mathbb{Z}$ ，则 $\lceil x \rceil = y + 1$ 。

$r \bmod m$ 唯一的整数 $r' \in \{0, 1, \dots, m-1\}$ 使得 $m \mid r - r'$ 。

$\text{BitRev}_7(r)$ 整数 r 的比特逆序。

$s \leftarrow x$ 在伪代码中，变量 s 被赋予表达式 x 的值。

$s \xleftarrow{\$} \mathbb{B}^\ell$ 在伪代码中， s 是从随机比特生成器中获取的 ℓ 个新鲜随机字节组成的数组。

$\perp$ 表示算法失败或无输出的符号。

k MLWR 采样数；

l MLWR 秘密的维度；

d 公钥压缩比特数。

d_1, d_2 密文压缩比特数。

η_s 私钥分布参数。

η_2 加密随机性分布参数。

β_η 参数为 η 的中心二项分布。

$\odot$ NTT 域中的乘法。

$\bmod q$ 模约简，结果落在 $\mathbb{Z}_q = \{0, \dots, q-1\}$ 中。

2.2 伪代码约定

本节概述本文档中用于描述算法的伪代码约定。所有算法都可访问两个全局整数常量： $n = 256$ 和 $q = 3329$ （以及 $n = 512$ 和 $q = 7681$ ）。此外还有若干全局整数变量： n 、 q 、 $k \times l$ 、 η_s 、 η_2 、 (d, d_1, d_2) 。所有其他变量为局部变量。当选定参数集后，这些全局变量被设置为特定值（见第 3.2 节）。

当本规范中的算法调用其他算法作为子程序时，所有参数（即输入）均按值传递。换句话说，创建输入的副本，并用该副本调用子程序。不存在“按引用传递”。

伪代码中的赋值使用符号“ $\leftarrow$ ”。例如，语句 $z \leftarrow y$ 表示变量 z 被赋予变量 y 的值。伪代码中的比较使用符号“ $==$ ”。例如，表达式 $x == w$ 是一个布尔值，当且仅当变量 x 和 w 具有相同的值时为真。

2.2.1 数据类型

对于表示算法输入或输出的变量，其数据类型（如比特、字节、比特数组）将在算法开头显式描述。对于伪代码中的大多数局部变量，数据类型可从上下文中轻松推断。对于所有其他变量，数据类型将在注释中声明。在单个算法中，变量的数据类型在首次使用时确定，且不会改变。变量名可以在不同算法中重复使用，包括具有不同数据类型的情况。

除了标准原子数据类型（如比特、字节）和数据结构（如数组）外，模 m 的整数（即 $\mathbb{Z}_m$ 的元素）也将作为抽象数据类型使用。每当向 $\mathbb{Z}_m$ 中的变量赋值时，隐含地进行模 m 约简。例如，对于 $z \in \mathbb{Z}_m$ 和整数 x 及 y ，语句

$$z \leftarrow x + y$$

表示 z 被赋予值 $x + y \bmod m$ 。伪代码不规定在实际实现中如何表示模 m 的整数或如何计算模约简。

2.2.2 整数数组的算术运算

本文档大量使用模 m 的整数数组（即 $\mathbb{Z}_m^\ell$ 的元素）。在典型情况下，相关取值为 $m = q = 3329$ ， $\ell = n = 256$ 或 $m = q = 7681$ ， $\ell = n = 512$ 。 $\mathbb{Z}_m^\ell$ 中数组的算术运算如下进行。设 $a \in \mathbb{Z}_m$ ， $X, Y \in \mathbb{Z}_m^\ell$ 。语句

$$Z \leftarrow a \cdot X \tag{2.2}$$

$$W \leftarrow X + Y \tag{2.3}$$

将产生两个数组 $Z, W \in \mathbb{Z}_m^\ell$ ，满足对所有 i 有 $Z[i] = a \cdot X[i]$ 和 $W[i] = X[i] + Y[i]$ 。 $\mathbb{Z}_m^\ell$ 中数组的乘法仅在 $m = q$ 且 $\ell = n$ 时有意义，此时它对应于特定环中的乘法。

2.2.3 代数对象的表示

COMPASS-KEM 中的一个核心操作是数论变换 (NTT)，它将特定环 R_q 中的多项式 f 映射到同构环 T_q 中的“NTT 表示” $\hat{f}$ 。本文档将使用模 q 的整数数组在伪代码中表示 R_q 和 T_q 的元素，具体如下。

R_q 的元素 f 是形如

$$f = f_0 + f_1X + \cdots + f_{n-1}X^{n-1} \in R_q \tag{2.4}$$

的多项式，在伪代码中由数组

$$(f_0, f_1, \dots, f_{n-1}) \in \mathbb{Z}_q^n \tag{2.5}$$

表示，其元素包含 f 的系数。通过符号重载，(2.5) 中的数组也记作 f 。数组 f 的第 i 个元素因此包含多项式 f 的第 i 个系数（即 $f[i] = f_i$ ）。

T_q 的元素（有时称为“NTT 表示”） $\hat{g}$ 是 $n/2$ 个多项式的元组，每个多项式的次数至多为一。具体来说，

$$\hat{g} = (\hat{g}_{0,0} + \hat{g}_{0,1}X, \hat{g}_{1,0} + \hat{g}_{1,1}X, \dots, \hat{g}_{n/2-1,0} + \hat{g}_{n/2-1,1}X) \in T_q. \tag{2.6}$$

这样的代数对象在伪代码中由数组

$$(\hat{g}_{0,0}, \hat{g}_{0,1}, \hat{g}_{1,0}, \hat{g}_{1,1}, \dots, \hat{g}_{n/2-1,0}, \hat{g}_{n/2-1,1}) \in \mathbb{Z}_q^n \tag{2.7}$$

表示。通过符号重载, (2.7) 中的数组也记作 $\hat{g}$ 。在此情况下, 数组元素与系数之间的映射为: 对所有 $i \in \{0, 1, \dots, n/2 - 1\}$, $\hat{g}[2i] = \hat{g}_{i,0}$, $\hat{g}[2i+1] = \hat{g}_{i,1}$ 。

多项式 $f \in R_q$ 与其 NTT 表示 $\hat{f} \in T_q$ 之间的转换通过算法 NTT 和 INTT 完成。这些算法作用于系数数组且满足 $\hat{f} = \text{NTT}(f)$ 和 $f = \text{INTT}(\hat{f})$ 。

2.2.4 多项式和 NTT 表示的算术运算

R_q 和 T_q 中的加法和标量乘法按坐标逐位进行。例如, 若 $a \in \mathbb{Z}_q$, $f \in R_q$, 则多项式 $a \cdot f \in R_q$ 的第 i 个系数等于 $a \cdot f_i \bmod q$ 。在伪代码中, R_q 和 T_q 的元素均由系数数组 (即 $\mathbb{Z}_q^n$ 的元素) 表示。因此加法和标量乘法通过对相应系数数组进行加法和标量乘法来执行, 如 (2.3) 和 (2.2) 所示。

R_q 和 T_q 中的乘法运算如下处理。出于效率考虑, 不使用 R_q 中的乘法。 T_q 中乘法的代数含义将在第 2.3.2 节详细讨论。具体来说, 对于 $f, g \in R_q$, 设 $\hat{f} = \text{NTT}(f)$ 和 $\hat{g} = \text{NTT}(g) \in \mathbb{Z}_q^n$ 为一对数组, 解释为 T_q 中的元素, 记作

$$\hat{h} \leftarrow \hat{f} \odot \hat{g}, \quad (2.8)$$

且对于 $h = \text{INTT}(\hat{h})$, 在环 R_q 中有 $h = f \cdot g$ 。

2.2.5 矩阵和向量

除了模 q 的整数数组外, 伪代码还将使用其元素本身为 $\mathbb{Z}_q^n$ 元素的数组。例如, 元素 $\mathbf{v} \in (\mathbb{Z}_q^n)^3$ 将是一个长度为三的数组, 其元素 $\mathbf{v}[0]$ 、 $\mathbf{v}[1]$ 和 $\mathbf{v}[2]$ 本身是 $\mathbb{Z}_q^n$ 的元素 (即数组)。可以将这些元素中的每一个视为表示 R_q 中的一个多项式, 因此 $\mathbf{v}$ 本身表示模 R_q^3 中的一个元素。

当数组用于表示元素属于 R_q 的矩阵和向量时, 它们将用粗体字母表示 (例如向量 $\mathbf{v}$, 矩阵 $\mathbf{A}$)。当数组用于表示元素属于 T_q 的矩阵和向量时, 它们将带有“帽子”符号 (例如 $\hat{\mathbf{v}}$ 和 $\hat{\mathbf{A}}$)。除非执行显式转置操作, 否则向量默认为列向量。因此, 可以将向量视为只有一列的矩阵的特例。

R_q 上的矩阵与 T_q 上的矩阵之间的转换按坐标逐位进行。例如, 若 $\mathbf{v} \in (\mathbb{Z}_q^n)^k$, 则语句

$$\hat{\mathbf{v}} \leftarrow \text{NTT}(\mathbf{v}) \quad (2.9)$$

将产生 $\hat{\mathbf{v}} \in (\mathbb{Z}_q^n)^k$, 满足对所有 i 有 $\hat{\mathbf{v}}[i] = \text{NTT}(\mathbf{v}[i])$ 。这涉及运行 NTT 共 k 次。

2.2.6 矩阵和向量的算术运算

下面描述如何对 R_q 和 T_q 上的矩阵 (以向量为特例) 执行算术运算。

加法和标量乘法按坐标逐分量进行, 因此 R_q 和 T_q 上矩阵的加法是直接的。对于 T_q , 标量乘法通过 (2.8) 进行。例如, 若 $\hat{f} \in \mathbb{Z}_q^n$ 且 $\hat{\mathbf{u}}, \hat{\mathbf{v}} \in (\mathbb{Z}_q^n)^k$, 则

$$\hat{\mathbf{w}} \leftarrow \hat{f} \odot \hat{\mathbf{u}} \quad (2.10)$$

$$\hat{\mathbf{z}} \leftarrow \hat{\mathbf{u}} + \hat{\mathbf{v}} \quad (2.11)$$

将产生 $\hat{\mathbf{w}}, \hat{\mathbf{z}} \in (\mathbb{Z}_q^n)^k$, 满足对所有 i 有 $\hat{\mathbf{w}}[i] = \hat{f} \odot \hat{\mathbf{u}}[i]$ 和 $\hat{\mathbf{z}}[i] = \hat{\mathbf{u}}[i] + \hat{\mathbf{v}}[i]$ 。

当需要对元素在 T_q 中的矩阵做乘法时, 通过使用标准矩阵乘法并将 T_q 中的乘法作为基础情况乘法来完成。若 $\hat{\mathbf{A}}$ 和 $\hat{\mathbf{B}}$ 是两个元素在 T_q 中的矩阵, 它们的矩阵乘积记作 $\hat{\mathbf{A}} \odot \hat{\mathbf{B}}$ 。下面给出涉及矩阵乘法的一些伪代码示例:

$$\hat{\mathbf{w}} \leftarrow \hat{\mathbf{A}} \odot \hat{\mathbf{u}} \quad \hat{\mathbf{w}}[i] = \sum_{j=0}^{k-1} \hat{\mathbf{A}}[i, j] \odot \hat{\mathbf{u}}[j] \quad (2.12)$$

$$\hat{\mathbf{y}} \leftarrow \hat{\mathbf{A}}^T \odot \hat{\mathbf{u}} \quad \hat{\mathbf{y}}[i] = \sum_{j=0}^{k-1} \hat{\mathbf{A}}[j, i] \odot \hat{\mathbf{u}}[j] \quad (2.13)$$

$$\hat{\mathbf{z}} \leftarrow \hat{\mathbf{u}}^T \odot \hat{\mathbf{v}} \quad \hat{\mathbf{z}} = \sum_{j=0}^{k-1} \hat{\mathbf{u}}[j] \odot \hat{\mathbf{v}}[j] \quad (2.14)$$

2.2.7 对数组应用算法: 示例

在前述示例中, $\mathbb{Z}_m$ 上的算术被扩展到 $\mathbb{Z}_m$ 上的数组的算术, 进而扩展到元素本身为 $\mathbb{Z}_m$ 上数组的矩阵的算术。类似地, 以给定数据类型为输入定义的算法将被应用于该数据类型上的数组和矩阵。当伪代码对数组或矩阵输入调用此类算法时, 隐含地将算法重复应用于输入的每个元素。

例如, 考虑函数 $\text{Comp}_d: \mathbb{Z}_q \rightarrow \mathbb{Z}_p$ (定义见第 2.3.3 节), 其中 $p = (q-1)/2^d$ 。可以通过以下语句对数组输入 $F \in \mathbb{Z}_q^n$ 调用该函数:

$$K \leftarrow \text{Comp}_d(F). \quad (2.15)$$

结果将是数组 $K \in \mathbb{Z}_p^n$, 满足对每个 $i \in \{0, 1, \dots, n-1\}$ 有 $K[i] = \text{Comp}_d(F[i])$ 。计算 (2.15) 涉及运行 Comp_d 算法 n 次。

再如, 考虑定义的 NTT 算法。它以数组 $f \in \mathbb{Z}_q^n$ (表示 R_q 的元素) 为输入, 输出另一个数组 $\hat{f} \in \mathbb{Z}_q^n$ (表示 T_q 的元素)。若通过以下伪代码语句对向量 $\mathbf{s} \in (\mathbb{Z}_q^n)^k$ (表示 R_q^k 的元素) 调用 NTT 算法:

$$\hat{\mathbf{s}} \leftarrow \text{NTT}(\mathbf{s}), \quad (2.16)$$

结果是一个向量 $\hat{\mathbf{s}} \in (\mathbb{Z}_q^n)^k$, 满足对所有 $i \in \{0, 1, \dots, k-1\}$ 有 $\hat{\mathbf{s}}[i] = \text{NTT}(\mathbf{s}[i])$ 。向量 $\hat{\mathbf{s}}$ 表示 T_q^k 的一个元素。计算 (2.16) 涉及运行 NTT 算法 k 次。

2.3 基本操作

2.3.1 哈希

展开矩阵 $\mathbf{A}$ 。 函数 ExpandA 将均匀种子 $\rho \in \{0, 1\}^n$ 映射到 NTT 域表示下的矩阵 $\mathbf{A} \in R_q^{k \times l}$ 。矩阵 $\mathbf{A}$ 仅用于乘法。因此, 为了更快的实现, 展开函数 ExpandA 不输出

$\mathbf{A} \in R_q^{k \times l} = (\mathbb{Z}_q[X]/(X^n + 1))^{k \times l}$, 而是输出 $\hat{\mathbf{A}} \in \mathbb{Z}_q^{k \times l \times n}$, 它被解释为 $\mathbf{A}$ 的 NTT 域表示。由于 $\mathbf{A}$ 需要均匀采样且 NTT 是同构, ExpandA 也需要在此表示中均匀采样。为了与 COMPASS-SIG 兼容, 如果实现产生的 NTT 向量顺序与我们的参考 NTT 不同, 则需要以非连续顺序采样系数。

采样向量 $\mathbf{s}$ 。 函数 Sample , 用于密钥生成中生成秘密向量, 给定分布 β_{η_s} , 将种子 σ 映射到 $\mathbf{s} \in \beta_{\eta_s}^\ell$ 。

哈希函数。 本文档还将使用三个哈希函数 H 、 J 和 G , 定义如下。

函数 H 和 J 各自接受一个可变长度输入, 产生一个 n 比特 ($n/8$ 字节) 输出。它们分别记作 $H: \mathbb{B}^* \rightarrow \mathbb{B}^{n/8}$ 和 $J: \mathbb{B}^* \rightarrow \mathbb{B}^{n/8}$ 。它们可以通过具有不同操作模式或不同前缀的安全哈希标准来实例化。

函数 G 接受一个可变长度输入, 产生两个 n 比特 ($n/8$ 字节) 输出。记作 $G: \mathbb{B}^* \rightarrow \mathbb{B}^{n/8} \times \mathbb{B}^{n/8}$ 。 G 的两个输出记作 $(a, b) \leftarrow G(c)$, 其中 $a, b \in \mathbb{B}^{n/8}$, $c \in \mathbb{B}^*$, 且 $G(c) = a \| b$ 。该函数可以通过具有与前两个函数不同操作模式或前缀的安全哈希标准来实例化。

2.3.2 NTT 域表示

数论变换 (NTT) 可以看作离散傅里叶变换的一种专门的精确版本。在 COMPASS-KEM 中, NTT 用于提高环 R_q 中乘法的效率, 其中 R_q 是环 $\mathbb{Z}_q[X]/(X^n + 1)$, 由形如 $f = f_0 + f_1X + \dots + f_{n-1}X^{n-1}$ (其中对所有 j 有 $f_j \in \mathbb{Z}_q$) 的多项式组成, 环运算定义为模 $X^n + 1$ 的算术运算。

环 R_q 同构于另一个环 T_q , 它是 $\mathbb{Z}_q$ 的 $n/2$ 个二次扩张的直和。NTT 是这两个环之间的一个计算高效的同构映射。当多项式 $f \in R_q$ 作为输入时, NTT 输出环 T_q 中的元素 $\hat{f} := \text{NTT}(f)$, 其中 $\hat{f}$ 称为 f 的 NTT 表示。同构性质意味着

$$f \cdot g = \text{INTT}(\hat{f} \odot \hat{g}), \quad (2.17)$$

其中 $\cdot$ 和 $\odot$ 分别表示 R_q 和 T_q 中的乘法。此外, 由于 T_q 是 $n/2$ 个环的乘积, 每个环由至多一次的多项式组成, 因此运算 $\odot$ 比运算 $\cdot$ 高效得多。基于这些原因, NTT 被视为 COMPASS-KEM 的一个组成部分, 而不仅仅是一项优化。

由于环 R_q 和 T_q 具有 $\mathbb{Z}_q$ 上的向量空间结构, 表示这两个环中任一个环的元素的最自然的抽象数据类型是 $\mathbb{Z}_q^n$ 。因此, NTT 和 INTT 的输入和输出选择长度- n 的模 q 整数数组作为数据结构。然而, 为了明确区分转换前后的代数对象, 算法以代数结构作为显式的输入和输出类型。

NTT 的数学结构。 对于两种参数集, 在 $\mathbb{Z}_q$ 中存在 n 次本原单位根但不存在 $2n$ 次本原单位根。我们取 ζ 为 $\mathbb{Z}_q$ 中的 n 次本原单位根。

定义 $\text{BitRev}_{\log n - 1}(i)$ 为对输入整数 $i \in \{0, \dots, n/2 - 1\}$ 的无符号 $\log n - 1$ 比特值进行比特逆序后所表示的整数。

多项式 $X^n + 1$ 模 q 可分解为 $n/2$ 个二次多项式如下:

$$X^n + 1 = \prod_{i=0}^{n/2-1} (X^2 - \zeta^{2 \text{BitRev}_{\log n-1}(i)+1}). \quad (2.18)$$

因此, $R_q := \mathbb{Z}_q[X]/(X^n + 1)$ 同构于 $\mathbb{Z}_q$ 的 $n/2$ 个二次扩张的直和, 记作 T_q 。具体来说, 该环具有如下结构:

$$T_q = \bigoplus_{i=0}^{n/2-1} \mathbb{Z}_q[X]/(X^2 - \zeta^{2 \text{BitRev}_{\log n-1}(i)+1}). \quad (2.19)$$

因此, 多项式 $f \in R_q$ 的 NTT 表示 $\hat{f} \in T_q$ 是由对应的至多一次的剩余组成的向量:

$$\hat{f} := (f \bmod (X^2 - \zeta^{2 \text{BitRev}_{\log n-1}(0)+1}), \dots, f \bmod (X^2 - \zeta^{2 \text{BitRev}_{\log n-1}(n/2-1)+1})). \quad (2.20)$$

如第 2.2.4 节所述, 本标准的算法将 $\hat{f}$ 表示为 n 个模 q 整数的数组。具体来说,

$$f \bmod (X^2 - \zeta^{2 \text{BitRev}_{\log n-1}(i)+1}) = \hat{f}[2i] + \hat{f}[2i+1]X, \quad (2.21)$$

其中 i 从 0 到 $n/2 - 1$ 。

T_q 中的乘法自然遵循如下定义: 对于 $\hat{f}, \hat{g} \in T_q, \hat{h} = \hat{f} \odot \hat{g} \in T_q$ 由元素 $\hat{h}[0], \dots, \hat{h}[n-1]$ 组成, 其中:

$$\hat{h}[2i] + \hat{h}[2i+1]X = (\hat{f}[2i] + \hat{f}[2i+1]X)(\hat{g}[2i] + \hat{g}[2i+1]X) \bmod (X^2 - \zeta^{2 \text{BitRev}_{\log n-1}(i)+1}) \quad (2.22)$$

其中 i 从 0 到 $n/2 - 1$ 。

2.3.3 压缩与解压缩

压缩函数 $\text{Comp}_d(x)$ 对于 $x \in \mathbb{Z}_q$, 以 d 为参数的压缩操作执行为:

1. 若 $x > 0$, 令 $x \leftarrow x - 1$;
2. 返回 $\lfloor x/2^d \rfloor$ 。

对于 $x \in R_q^k$, Comp_d 分别应用于 x 中每个元素的每个系数。

不难看出, 若 $2^d \mid q-1$, 则 $\text{Comp}_d(x) = \lfloor \frac{p}{q} \cdot x \rfloor$ (其中 $p = (q-1)/2^d$), 这符合困难假设 MLWR (第 3.1 节) 中舍入运算的定义。

解压缩函数 $\text{Decomp}_d(x)$ 对于 $x \in \mathbb{Z}_q$, 以 d 为参数的解压缩操作返回 $x \cdot 2^d + 2^{d-1}$ 。

对于 $x \in R_q^k$, Decomp_d 分别应用于 x 中每个元素的每个系数。满足以下条件: $|x - \text{Decomp}_d(\text{Comp}_d(x))| \leq 2^{d-1}$ 。

需要注意, Comp_d 和 Decomp_d 均不能应用于 NTT 域。

2.4 COMPASS-PKE 算法

本节描述组件方案 COMPASS-PKE。COMPASS-PKE 不允许以独立方式使用。它仅作为子程序集合，供正式方案 COMPASS-KEM 的算法使用。

COMPASS-PKE 包含三个算法：

1. 密钥生成 (COMPASS-PKE.KeyGen)。
2. 加密 (COMPASS-PKE.Encrypt)。
3. 解密 (COMPASS-PKE.Decrypt)。

当 COMPASS-PKE 作为 COMPASS-KEM 的一部分实例化时，COMPASS-PKE 继承为 COMPASS-KEM 选择的参数集。每个参数集为每个参数指定数值。参数和参数集在第 3.2 节中描述。

本节中的算法不执行任何输入检查，因为它们仅作为主 COMPASS-KEM 算法的子程序被调用。COMPASS-KEM 的算法本身根据需要执行输入检查。

COMPASS-PKE 的每个算法都附有简短的非正式文字描述。为简单起见，该描述以元素属于 R_q 的向量和矩阵来书写。在实际算法中，大多数计算在 NTT 域中进行以提高乘法效率。相关向量和矩阵的元素将属于 T_q 。

2.4.1 密钥生成 (KeyGen)

密钥生成算法 COMPASS-PKE.KeyGen 输出一个加密密钥 pk 和一个解密密钥 sk 。如同公钥加密的典型情况，加密密钥可以公开，而解密密钥和随机性必须保持私密。实际上，COMPASS-PKE 的加密密钥将作为 COMPASS-KEM 的封装密钥（见下文的 COMPASS-KEM.KeyGen），因此可以公开。同时，COMPASS-PKE.KeyGen 的解密密钥和种子必须保持私密，因为它们可用于在 COMPASS-KEM 中执行解封装。

在 COMPASS-PKE.KeyGen 的第 3 步中生成的矩阵 $\hat{\mathbf{A}}$ 可以按规范存储。这允许后续操作直接使用 $\hat{\mathbf{A}}$ ，而无需从公开种子 ρ 重新展开它。

COMPASS-PKE.KeyGen 的解密密钥是 R_q 中元素的一个长度为 k 的向量 $\mathbf{s}$ （即 $\mathbf{s} \in R_q^k$ ）。总体而言， $\mathbf{s}$ 是一组秘密变量，而加密密钥是关于秘密变量 $\mathbf{s}$ 的一组压缩线性方程 $(\mathbf{A}, \text{Comp}_d(\mathbf{A}\mathbf{s}))$ 。矩阵 $\mathbf{A}$ 的行构成方程系数。该矩阵使用 XOF 伪随机生成。完成 $\mathbf{A}$ 和 $\mathbf{s}$ 的生成后，计算加密密钥的最后部分 $\mathbf{t} = \text{Comp}_d(\mathbf{A}\mathbf{s})$ 。结果被适当编码为字节数组并输出。

在 COMPASS-PKE.KeyGen 中，参数集的选择影响秘密 $\mathbf{s}$ 的长度（通过参数 k ），进而影响伪随机矩阵 $\mathbf{A}$ 的大小。参数集的选择还影响用于采样 $\mathbf{s}$ 各元素的分布（通过参数 η_s ）。

Algorithm 1 COMPASS-PKE.KeyGen()**输入：** 无

1. $\iota \xleftarrow{\$} \mathbb{B}^{n/8}$ ▷ 随机采样 n 比特初始种子
2. $(\rho, \sigma) \leftarrow \mathbf{G}(\iota)$ ▷ 通过哈希函数 G 派生 ρ 和 σ
3. $\hat{\mathbf{A}} \leftarrow \text{ExpandA}(\rho) \in R_q^{k \times l}$ ▷ 使用 ρ 在 NTT 域中构建矩阵
4. $\mathbf{s} \leftarrow \text{Sample}(\beta_{\eta_s}, \sigma) \in R^l$ ▷ 使用 σ 采样私钥向量 $\mathbf{s}$
5. $\hat{\mathbf{s}} \leftarrow \text{NTT}(\mathbf{s})$ ▷ 将私钥变换到 NTT 域
6. $\mathbf{t}' \leftarrow \text{INTT}(\hat{\mathbf{A}} \odot \hat{\mathbf{s}})$ ▷ 在 NTT 域中相乘并变换回原域
7. $\mathbf{t} \leftarrow \text{Comp}_d(\mathbf{t}')$ ▷ 压缩公钥
8. $pk \leftarrow (\rho, \mathbf{t})$
9. $sk \leftarrow (\iota, \hat{\mathbf{s}})$

输出： 公私钥对 (pk, sk) **2.4.2 加密 (Encrypt)**

COMPASS-PKE 的加密算法 COMPASS-PKE.Encrypt 以加密密钥 ek_{PKE} 、一个 $n/8$ 字节的明文 $\mathbf{m}$ 和随机性 $\mathbf{r}$ 为输入，产生单个密文 c 作为输出。

在 COMPASS-PKE.Encrypt 的第 1 步中生成的矩阵 $\hat{\mathbf{A}}$ 可以按规范存储。这允许后续操作直接使用 $\hat{\mathbf{A}}$ ，而无需从公开种子 ρ 重新展开它。

算法 COMPASS-PKE.Encrypt 首先从加密密钥中提取向量 $\mathbf{t}$ 和种子。然后展开种子以重新生成矩阵 $\mathbf{A}$ ，方式与 COMPASS-PKE.KeyGen 中相同。若 $\mathbf{t}$ 和 $\mathbf{A}$ 正确地由 COMPASS-PKE.KeyGen 生成的加密密钥中派生，则它们等于 COMPASS-PKE.KeyGen 中相应的值。

回顾密钥生成的描述， $(\mathbf{A}, \text{Decomp}_d(\mathbf{t}))$ (其中 $\mathbf{t} = \text{Comp}_d(\mathbf{A}\mathbf{s})$) 可以看作关于秘密变量 $\mathbf{s}$ 的一个压缩线性方程组。可以在不知道 $\mathbf{s}$ 的情况下，通过选取该系统 $(\mathbf{A}, \text{Decomp}_d(\mathbf{t}))$ 中带噪方程的随机线性组合，生成一个额外的关于相同秘密变量的带噪线性方程。然后，可以将信息编码到这样一个组合方程的“常数项”中。拥有 $\mathbf{s}$ 的一方可以解出此信息。例如，可以通过决定是否显著改变常数项来编码单个比特：使方程几乎正确对应于解密比特值 0，或者使方程远不正确对应于解密比特值 1。在 COMPASS-PKE 的情况下，常数项是具有 n 个系数的多项式，因此可以编码更多信息：每个系数编码一个比特。

为此，COMPASS-PKE.Encrypt 首先生成向量 $\mathbf{r} \in R_q^k$ 。然后计算“新的压缩方程”，即 $(\text{Comp}_{d_1}(\mathbf{A}^T \mathbf{r}), \text{Comp}_{d_2}(\mathbf{t}^T \mathbf{r}))$ 。然后将输入消息 m 的适当编码加到该对的后者上。最后，将结果对 (c_1, c_2) 序列化为字节数组，作为密文输出。

Algorithm 2 COMPASS-PKE.Encrypt(pk, m, r)**输入:** $pk = (\rho, \mathbf{t})$; n 比特消息 $m \in \{0, 1\}^n$; 随机种子 $r \in \mathbb{B}^{n/8}$

1. $\hat{\mathbf{A}} \leftarrow \text{ExpandA}(\rho) \in R_q^{k \times l}$ ▷ 从 $pk.seed$ 恢复 $\hat{\mathbf{A}}$
2. $\mathbf{t}' \leftarrow \text{Decomp}_d(\mathbf{t})$ ▷ 解压缩公钥
3. $\mathbf{r} \leftarrow \text{Sample}(\beta_{\eta_2}, r) \in R^k$ ▷ 从 β_{η_2} 采样 $n \times k$ 个元素
4. $\hat{\mathbf{r}} \leftarrow \text{NTT}(\mathbf{r})$
5. $\mathbf{c}'_1 \leftarrow \text{INTT}(\hat{\mathbf{A}}^T \odot \hat{\mathbf{r}})$
6. $c'_2 \leftarrow \text{INTT}(\text{NTT}(\mathbf{t}')^T \odot \hat{\mathbf{r}})$
7. $c'_2 \leftarrow c'_2 + m \cdot \lfloor q/2 \rfloor \pmod{q}$ ▷ 将消息编码到高位
8. $\mathbf{c}_1 \leftarrow \text{Comp}_{d_1}(\mathbf{c}'_1)$
9. $c_2 \leftarrow \text{Comp}_{d_2}(c'_2)$

输出: $c = (c_1, c_2)$ **2.4.3 解密 (Decrypt)**

COMPASS-PKE 的解密算法 COMPASS-PKE.Decrypt 以解密密钥 sk 和密文 c 为输入, 无需随机性, 输出明文 m 。

算法 COMPASS-PKE.Decrypt 首先从密文 c 中恢复对 $(\mathbf{c}'_1, c'_2)$ 。这里, 可以将 $\mathbf{c}'_1$ 视为方程的系数, c'_2 视为常数项。解密密钥 sk 包含秘密变量向量 $\mathbf{s}$ 。因此解密算法可以使用解密密钥计算真实的常数项 $v = \mathbf{s}^T \mathbf{c}'_1$, 并计算 $c'_2 - v$ 。解密算法最后从 $c'_2 - v$ 中解码出明文消息 m 并输出 m 。

Algorithm 3 COMPASS-PKE.Decrypt(sk, ct)**输入:** $sk = (z, \hat{\mathbf{s}})$; 密文 $ct = (\mathbf{c}_1, c_2)$

1. $\mathbf{c}'_1 \leftarrow \text{Decomp}_{d_1}(\mathbf{c}_1)$
2. $c'_2 \leftarrow \text{Decomp}_{d_2}(c_2)$
3. $u \leftarrow c'_2 - \text{INTT}(\text{NTT}(\mathbf{c}'_1)^T \odot \hat{\mathbf{s}}) \pmod{q}$
4. **for** $i \leftarrow 0$ **to** $n - 1$ **do**
5. **if** $\lfloor q/4 \rfloor \leq u[i] < \lfloor 3q/4 \rfloor$ **then**
6. $m[i] \leftarrow 1$
7. **else**
8. $m[i] \leftarrow 0$
9. **end if**
10. **end for**

输出: 消息 $m \in \{0, 1\}^n$

2.5 主算法

本节规定三个算法：COMPASS-KEM.KeyGen、COMPASS-KEM.Encaps 和 COMPASS-KEM.Decaps。本节中的算法使用参数 n 、 q 、 $k \times l$ 、 (d, d_1, d_2) 。它们调用的子程序额外使用参数 η_s 和 η_2 。

2.5.1 密钥生成

算法 COMPASS-KEM.KeyGen 生成一个封装密钥和一个解封装密钥。

COMPASS-KEM.KeyGen 的核心子程序是 COMPASS-PKE 的密钥生成算法。封装密钥即 COMPASS-PKE 的加密密钥。解封装密钥由 COMPASS-PKE 的解密密钥、封装密钥、封装密钥的哈希值以及一个随机的 $n/8$ 字节值组成。该随机值将用于解封装算法的“隐式拒绝”机制。

Algorithm 4 COMPASS-KEM.KeyGen()

输入： 无

1. $(ek_{PKE}, dk_{PKE}) \leftarrow \text{COMPASS-PKE.KeyGen}()$ ▷ 运行 COMPASS-PKE 的密钥生成
2. $z \xleftarrow{\$} \mathbb{B}^{n/8}$
3. $ek \leftarrow ek_{PKE}$ ▷ KEM 封装密钥即 PKE 加密密钥
4. $dk \leftarrow (dk_{PKE} \| ek \| H(ek) \| z)$ ▷ KEM 解封装密钥包含 PKE 解密密钥
5. **返回** (ek, dk)

输出： 封装密钥 ek ，解封装密钥 dk

2.5.2 封装

算法 COMPASS-KEM.Encaps 以封装密钥为输入，输出密文和共享密钥。

COMPASS-KEM.Encaps 的核心子程序是 COMPASS-PKE 的加密算法，用于将随机值 m 加密为密文 c 。共享秘密密钥 K 的副本和加密期间使用的随机性通过哈希从 m 和封装密钥 ek 派生。具体来说，对 ek 应用 H ，将结果与 m 拼接，然后使用 G 进行哈希。最后，算法输出共享秘密密钥 K 和密文 c 。

Algorithm 5 COMPASS-KEM.Encaps(ek)

输入： 封装密钥 ek

1. $m \xleftarrow{\$} \mathbb{B}^{n/8}$
2. $(K, r) \leftarrow G(m \| H(ek))$ ▷ 派生共享秘密密钥 K 和随机性 r
3. $c \leftarrow \text{COMPASS-PKE.Encrypt}(ek, m, r)$ ▷ 使用 COMPASS-PKE 以随机性 r 加密 m
4. **返回** (K, c)

输出： 共享秘密密钥 $K \in \mathbb{B}^{n/8}$ ，密文 c

2.5.3 解封装

算法 COMPASS-KEM.Decaps 以解封装密钥和密文为输入，不使用任何随机性，输出共享秘密密钥。

算法 COMPASS-KEM.Decaps 首先解析 COMPASS-KEM 解封装密钥 dk 的各分量。这些分量是 COMPASS-PKE 的（加密密钥，解密密钥）对、一个哈希值 h 和一个随机值 z 。然后使用 COMPASS-PKE 的解密密钥解密输入密文 c 得到明文 m' 。解封装算法接着以与封装中相同的方式重新加密 m' 并计算候选共享秘密密钥 K' 。具体来说， K' 和加密随机性 r' 通过对 m' 和 COMPASS-PKE 的加密密钥进行哈希来计算，并使用随机性 r' 加密 m' 生成密文 c' 。最后，解封装检查结果密文 c' 是否与提供的密文 c 匹配。若不匹配，算法执行“隐式拒绝”：将 K' 的值更改为将 c 与存储在 COMPASS-KEM 秘密密钥中的随机值 z 一起哈希的结果。无论哪种情况，解封装都输出结果共享秘密密钥 K' 。

在第 6 步中计算的“隐式拒绝”标志（通过比较 c 和 c' ）是一段秘密的中间数据。该标志必须在 COMPASS-KEM.Decaps 终止之前销毁。特别地，不允许以任何形式返回该标志的值作为输出。

Algorithm 6 COMPASS-KEM.Decaps(dk, c)

输入： 解封装密钥 dk ，密文 c

1. $(dk_{PKE}, ek_{PKE}, h, z) \leftarrow dk$ ▷ 从解封装密钥中提取所有分量
2. $m' \leftarrow \text{COMPASS-PKE.Decrypt}(dk_{PKE}, c)$ ▷ 解密密文
3. $(K', r') \leftarrow \text{G}(m' || h)$ ▷ 派生共享秘密和随机性
4. $\bar{K} \leftarrow \text{J}(z || c)$ ▷ 计算隐式拒绝密钥
5. $c' \leftarrow \text{COMPASS-PKE.Encrypt}(ek_{PKE}, m', r')$ ▷ 重新加密以验证 c
6. **if** $c \neq c'$ **then**
7. $K' \leftarrow \bar{K}$ ▷ 若密文不匹配则隐式拒绝
8. **end if**
9. **返回** K'

输出： 共享秘密密钥 $K' \in \mathbb{B}^{n/8}$

3. 设计原理

3.1 安全基础

本文档中给出的密码构造基于普通的模舍入学习 (MLWR) 问题和一个名为有偏 MLWR (biased MLWR) 的新困难问题。我们首先定义舍入运算：对于 $x \in \mathbb{Z}_q$ ， $\lfloor x \rfloor_p$ 定义为 $\lfloor \frac{p}{q} \cdot x \rfloor$ 。对于 $2^d \mid q - 1$ ，可以看出 $\text{Comp}_d(x) = \lfloor x \rfloor_p$ （其中 $p = (q - 1)/2^d$ ）。

定义 1. (MLWR [DKRV18]) 设 $q \geq p \geq 2$ ，公开项 $\mathbf{A} \xleftarrow{\$} R_q^{k \times l}$ ，秘密项 $\mathbf{s} \xleftarrow{\$} R_q^l$ 。则

$R_q^{k \times l} \times R_p^k$ 上的 $MLWR_{n,q,p,k,l}$ 分布定义为

$$(\mathbf{A}, \mathbf{b} = \lfloor \mathbf{A}\mathbf{s} \rfloor_p).$$

(判定性) $MLWR_{n,q,p,k,l}$ 问题定义为区分 $MLWR_{n,q,p,k,l}$ 分布与 $(\mathbf{A}, \lfloor \mathbf{u} \rfloor_p)$, 其中 $(\mathbf{A}, \mathbf{u})$ 是 $R_q^{k \times l} \times R_q^k$ 上的均匀随机分布。

定义 2. (有偏 $MLWR$) 设 $q \geq p \geq 2$, 公开项 $\tilde{\mathbf{A}} \xleftarrow{\$} R_q^{k \times l}, \mathbf{t} \xleftarrow{\$} R_q^k$, 秘密项 $\mathbf{s} \xleftarrow{\$} R_q^k$. 设 $\mathbf{t}' = \text{Decomp}_d(\text{Comp}_d(\mathbf{t}))$, $\mathbf{A} = [\tilde{\mathbf{A}}|\mathbf{t}']$. 则 $R_q^{k \times (l+1)} \times R_p^{l+1}$ 上的有偏 $MLWR_{n,q,p,d,k,l}$ 分布定义为

$$(\mathbf{A}, \mathbf{b} = \lfloor \mathbf{s}^T \mathbf{A} \rfloor_p).$$

(判定性) 有偏 $MLWR_{n,q,p,d,k,l}$ 问题定义为区分有偏 $MLWR_{n,q,p,d,k,l}$ 分布与 $R_q^{k \times (l+1)} \times R_p^{l+1}$ 上的分布 $(\mathbf{A}, \lfloor \mathbf{u} \rfloor_p)$, 其中 $\mathbf{u} \xleftarrow{\$} R_q^{l+1}$.

有必要指出, SABER [DKRV18] 作为唯一具有正式证明的基于 $MLWR$ 的方案, 其安全归约存在缺陷。为了修复这一问题, 不可避免地需要在安全证明中引入有偏 $MLWR$ 作为一个新的困难问题。如下文所述, 这一新问题不影响具体安全评估。

普通 $MLWR$ 的困难性可以通过工作 [LW20] 归约到 $MLWE$ (或 $RLWE$) 的困难性, 然而有偏 $MLWR$ 的理论困难性并不显然。在本部分中, 我们给出从 $MLWE$ 到有偏 $MLWR$ 的归约, 以证明引入这一非标准 $MLWR$ 变体并不降低我们方案的可证明安全性。

定理 1. 设 $q \geq p \geq 2$. 存在从上文定义的 $MLWE$ 到有偏 $MLWR$ 的多项式时间归约。证明. 目标是证明有偏 $MLWR$ 样本中的元素 $\mathbf{b} = \lfloor \mathbf{s}^T \mathbf{A} \rfloor_p$ 与均匀随机元素不可区分。

首先, 根据压缩和解压缩操作的定义, 公开项 $\mathbf{A}$ 可重写为 $\mathbf{A} = [\tilde{\mathbf{A}}|\mathbf{t} + \mathbf{z}]$, 其中 $\|\mathbf{z}\|_\infty \leq 2^{d-1}$. 则元素 $\mathbf{b} = \lfloor \mathbf{s}^T \mathbf{A} \rfloor_p = \lfloor [\mathbf{s}^T \tilde{\mathbf{A}}|\mathbf{s}^T \mathbf{t} + \mathbf{s}^T \mathbf{z}] \rfloor_p$.

第二, 由 $MLWE$ 假设, 均匀随机元素 $\tilde{\mathbf{A}}$ 和 $\mathbf{t}$ 可替换为 $\mathbf{C}\mathbf{B} + \mathbf{F}$ 和 $\mathbf{c}^T \mathbf{B} + \mathbf{f}$, 其中 $\mathbf{C}, \mathbf{c}$ 和 $\mathbf{F}, \mathbf{f}$ 具有小范数。因此, 元素 $\mathbf{b}$ 可进一步重写为 $\mathbf{b} = \lfloor [\mathbf{s}^T \mathbf{C}\mathbf{B} + \mathbf{s}^T \mathbf{F} | (\mathbf{c}\mathbf{s})^T \mathbf{B} + \mathbf{s}^T \mathbf{f} + \mathbf{s}^T \mathbf{z}] \rfloor_p$.

接下来, 我们需要证明 $\mathbf{s}$ 的剩余最小熵仍足以进行随机提取, 即

$$H_\infty \left(\mathbf{s} | \bar{\mathbf{A}}, \lfloor [\mathbf{s}^T \mathbf{C}\mathbf{B} + \mathbf{s}^T \mathbf{F} | (\mathbf{c}\mathbf{s})^T \mathbf{B} + \mathbf{s}^T \mathbf{f} + \mathbf{s}^T \mathbf{z}] \rfloor_p \right)$$

足够大, 其中为简单起见设 $\bar{\mathbf{A}} = [\mathbf{C}\mathbf{B} + \mathbf{F} | \mathbf{c}^T \mathbf{B} + \mathbf{f} + \mathbf{z}]$. 为此, 我们将秘密 $\mathbf{s}$ 分为两部分: $\mathbf{s} = \mathbf{s}_1 + \mathbf{s}_2$, 其中 $\mathbf{s}_1$ 是均匀随机的, $\mathbf{s}_2$ 是 γ 有界的, 并定义集合 I 为 $\mathbf{s}^T \bar{\mathbf{A}}$ 中满足 $\lfloor \mathbf{s}^T \bar{\mathbf{A}} \rfloor_p \neq \lfloor [\mathbf{s}^T \mathbf{C}\mathbf{B} | (\mathbf{c}\mathbf{s})^T \mathbf{B}] \rfloor_p$ 的指标及相应系数。则通过与 [LW20] 类似的论证 (基于 Chernoff 界的论证), I 的大小是有界的。现在, $\lfloor \mathbf{s}^T \bar{\mathbf{A}} \rfloor_p$ 可由项 $\mathbf{s}^T \mathbf{C}, \mathbf{C}, \mathbf{B}, \mathbf{c}\mathbf{s}^T, \mathbf{c}$ 和 I 重构。因此

$$\begin{aligned} H_\infty (\mathbf{s} | \bar{\mathbf{A}}, \lfloor \mathbf{s}^T \bar{\mathbf{A}} \rfloor_p) &= H_\infty \left(\mathbf{s} | \bar{\mathbf{A}}, \lfloor [\mathbf{s}^T \mathbf{C}\mathbf{B} + \mathbf{s}^T \mathbf{F} | (\mathbf{c}\mathbf{s})^T \mathbf{B} + \mathbf{s}^T \mathbf{f} + \mathbf{s}^T \mathbf{z}] \rfloor_p \right) \\ &\geq H_\infty (\mathbf{s} | \mathbf{s}_2^T \mathbf{C}, \mathbf{C}, \mathbf{B}, (\mathbf{c}\mathbf{s}_2)^T, \mathbf{c}, \mathbf{s}_1 \bar{\mathbf{A}}, I) \\ &\geq H_\infty (\mathbf{s} | \mathbf{s}_1) - (n + \lambda) \log q \\ &\geq kn \log(2\gamma) - (n + \lambda) \log q. \end{aligned}$$

通过适当设置参数, 即 $kn \log(2\gamma) - (n + \lambda) \log q \geq n \log(q) + 2\lambda$, 可以应用基于环的剩余哈希引理 (参考文献 [LW20]) 证明

$$\left(\begin{bmatrix} \lfloor \bar{\mathbf{A}} \rfloor_p \\ \lfloor \mathbf{a} \rfloor_p \end{bmatrix}, \begin{bmatrix} \lfloor \mathbf{s}^T \bar{\mathbf{A}} \rfloor_p \\ \lfloor \langle \mathbf{a}, \mathbf{s} \rangle \rfloor_p \end{bmatrix} \right) \stackrel{s}{\approx} \left(\begin{bmatrix} \lfloor \bar{\mathbf{A}} \rfloor_p \\ \lfloor \mathbf{a} \rfloor_p \end{bmatrix}, \begin{bmatrix} \lfloor \mathbf{s}^T \bar{\mathbf{A}} \rfloor_p \\ \lfloor u \rfloor_p \end{bmatrix} \right).$$

精确的统计距离以 $2^{-\lambda}$ 为界。最后, 我们通过 **MLWE** 将 $\bar{\mathbf{A}}$ 替换为 $\mathbf{A}$ 得到:

$$\left(\begin{bmatrix} \lfloor \bar{\mathbf{A}} \rfloor_p \\ \lfloor \mathbf{a} \rfloor_p \end{bmatrix}, \begin{bmatrix} \lfloor \mathbf{s}^T \bar{\mathbf{A}} \rfloor_p \\ \lfloor u \rfloor_p \end{bmatrix} \right) \stackrel{c}{\approx} \left(\begin{bmatrix} \lfloor \mathbf{A} \rfloor_p \\ \lfloor \mathbf{a} \rfloor_p \end{bmatrix}, \begin{bmatrix} \lfloor \mathbf{s}^T \mathbf{A} \rfloor_p \\ \lfloor u \rfloor_p \end{bmatrix} \right).$$

通过简单的混合论证, 我们可以进一步证明期望的命题:

$$(\mathbf{A}, \lfloor \mathbf{s}^T \mathbf{A} \rfloor_p) \stackrel{c}{\approx} (\mathbf{A}, \lfloor \mathbf{u} \rfloor_p).$$

□

3.2 参数选择

为适应不同的安全需求, 我们提供了四组参数集, 分别针对 128、256、384 和 512 比特安全性。所有参数集均使用素数模数 q 、环次数 n 为 256 或 512, 以及适当的矩阵维度 ($k \times l$)。对每个参数集, 秘密向量 $\mathbf{s}$ 和加密随机性 $\mathbf{r}$ 的系数取自参数为 $\{2, 3, 4\}$ 的中心二项分布 (即 $\eta_s = \eta_2 \in \{2, 3, 4\}$), 这既确保了方案的安全级别, 也保证了解密正确性。参数 d, d_1, d_2 定义了用于压缩和解压缩的比特数, 以减小公钥和密文规模, 其中对于不同安全级别 $d = d_1 = 2$ 且 $d_2 \in \{6, 8\}$ 。所有参数都经过精心选择, 以保证适中的公钥和密文规模、解密正确性, 以及抵御已知格攻击 (如原始攻击和对偶攻击) 的能力。具体数值列于下表中。

表 1: 安全级别与 COMPASS-KEM 参数映射

级别	n	q	p	$k \times l$	η_s	(d, d_1, d_2)	目标安全性
级别 1	256	3329	832	2×2	3	(2, 2, 8)	128 bit
级别 2	256	3329	832	4×4	2	(2, 2, 6)	256 bit
级别 3	512	7681	1920	3×3	3	(2, 2, 8)	384 bit
级别 4	512	7681	1920	4×4	4	(2, 2, 6)	512 bit

下面给出 COMPASS-KEM 在不同安全级别下的密钥和密文规模, 以及与 ML-KEM 在相似安全级别下的对比。可以看出, 在 128 和 256 比特安全级别下, 我们的方案在密钥和密文规模上均优于 ML-KEM。

表 2: COMPASS-KEM 的密钥和密文规模 (字节)

算法名称	安全级别	公钥	私钥	密文
COMPASS-KEM-128	128 bit	672	1504	768
ML-KEM-512	128 bit	800	1632	768
COMPASS-KEM-256	256 bit	1312	2912	1472
ML-KEM-1024	256 bit	1568	3168	1568
COMPASS-KEM-384	384 bit	2144	4704	2432
COMPASS-KEM-512	512 bit	2848	6240	3264

3.3 解密失败率分析

下面分析 COMPASS-KEM 方案的解密失败率。COMPASS-KEM 方案与子程序 COMPASS-PKE 具有相同的失败率。在 COMPASS-PKE 解密算法的背景下，以下等式在模 q 下成立：

$$\begin{aligned}
 u &= c'_2 - \mathbf{c}'_1 \cdot \mathbf{s} = m \cdot \frac{q-1}{2} + (\mathbf{A} \cdot \mathbf{s} + \mathbf{e}_t)^T \cdot \mathbf{r} + e_2 - (\mathbf{A}^T \cdot \mathbf{r} + \mathbf{e}_1)^T \cdot \mathbf{s} \\
 &= m \cdot \frac{q-1}{2} + \mathbf{e}_t^T \cdot \mathbf{r} + e_2 - \mathbf{e}_1^T \cdot \mathbf{s},
 \end{aligned}$$

其中 $\mathbf{e}_t$ 、 $\mathbf{e}_1$ 和 e_2 是由压缩引起的误差。因此，若 $e_{\text{total}} := \mathbf{e}_t^T \cdot \mathbf{r} + e_2 - \mathbf{e}_1^T \cdot \mathbf{s}$ 的无穷范数小于 $\frac{q-1}{4}$ ，解密即为正确。

为评估解密失败率，我们对相关分布建模： $\mathbf{s} \leftarrow \beta_{\eta_s}$, $\mathbf{r} \leftarrow \beta_{\eta_2}$, $\mathbf{e}_t \leftarrow \{\text{Decomp}_d(\text{Comp}_d(x)) - x \mid x \xleftarrow{\$} \mathbb{Z}_q\}$, $\mathbf{e}_1 \leftarrow \{\text{Decomp}_{d_1}(\text{Comp}_{d_1}(x)) - x \mid x \xleftarrow{\$} \mathbb{Z}_q\}$, $e_2 \leftarrow \{\text{Decomp}_{d_2}(\text{Comp}_{d_2}(x)) - x \mid x \xleftarrow{\$} \mathbb{Z}_q\}$ 。利用中心极限定理，我们进一步将乘积项 $\mathbf{e}_t^T \cdot \mathbf{r} - \mathbf{e}_1^T \cdot \mathbf{s}$ 中的所有坐标建模为高斯分布 $\mathcal{N}(0, n \cdot (\sigma_{\mathbf{e}_t}^2 \sigma_{\mathbf{r}}^2 + \sigma_{\mathbf{e}_1}^2 \sigma_{\mathbf{s}}^2))$ 。 e_2 的分布由离散概率分布表刻画。我们可以首先计算 e_{total} 的单个坐标的绝对值超过 $\frac{q-1}{4}$ 的概率，然后通过联合界推导出整体解密失败率。COMPASS-PKE (即 COMPASS-KEM) 所有参数集的解密失败率如表3所示。

表 3: 解密失败率

安全级别	解密失败率
级别 1	$2^{-189.27}$
级别 2	$2^{-181.13}$
级别 3	$2^{-404.51}$
级别 4	$2^{-250.31}$

我们注意到，提案中最大解封装调用次数以 2^{80} 为界，因此所有级别的解密失败率远低于调用次数，这意味着利用解密失败几乎是不可能的。

3.4 其他考虑因素

已有若干工作 [YXP20, XWG24] 声称可以对后量子 KEM（尤其是密钥生成算法）应用密码后门攻击。对于 [YXP20] 攻击，我们将密钥生成中使用的主种子 ι 包含在秘密密钥中，因此该攻击可被秘密密钥持有者检测到：首先使用 ι 生成公开种子 ρ ，然后检查 ρ 是否在公钥中被篡改。

在 Xia 等人 [XWG24] 的后量子 Kyber-KEM 后门攻击中，攻击者将 McEliece 密文嵌入公钥分量 $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}$ 的最低有效位 (LSB)，随后从公钥中提取这些 LSB 以恢复秘密种子。该攻击依赖两个结构性质：(1) 公钥 LWE 样本的 LSB 必须保持可用；(2) 密钥生成算法必须包含一个显式的误差项，其奇偶性可被控制，同时保持其规定分布。

我们的 KEM 设计消除了这两个攻击面。首先，公钥分量不以完整值 $\mathbf{t}'$ 发布。相反，密钥生成仅输出 $\mathbf{t} = \text{Comp}_d(\mathbf{t}')$ ，因此每个系数的 d 个 LSB 在序列化之前即被丢弃。解压缩过程仅返回一个规范的四舍五入代表值，并不重建原始的 LSB。因此，公钥不为嵌入或恢复辅助密文提供可靠的低比特信道。其次，该方案遵循 MLWR 范式：公钥噪声由确定性舍入产生，而非由独立采样的误差向量产生。因此，敌手密钥生成器无法通过选择显式误差项的奇偶性来编码后门密文，同时保持规定的密钥分布不变。因此，[XWG24] 中基于 LSB 的 Kyber-KEM 后门不适用于我们的构造。

3.5 差异性声明

我们的方案紧密遵循已得到充分发展的 [LP11] 模格范式，这意味着我们的方案与同一范式下的其他方案（典型的如 ML-KEM 和 SABER）具有相似的结构。然而，我们的方案与二者存在显著差异。

与 ML-KEM 相比，我们的方案基于不同的困难问题，即 MLWR 及其变体，因此 ML-KEM 中的误差采样被更快的压缩算法所替代，减少了公钥规模以及密钥生成和封装算法的总时间开销，同时也规避了基于 LWE 方案中潜在的知识产权问题。

与 SABER 相比，我们的方案使用素模数而非 2 的幂次模数上的 MLWR。素模数上的 MLWR 可以归约到 MLWE，因而享有从标准格问题的最坏情况到平均情况的归约，而 2 的幂次模数上的 MLWR 不具备类似的归约。此外，使用素模数允许我们使用 NTT 进行多项式乘法，因此在时间复杂度上优于 SABER。

4. 安全声明与分析

4.1 理论安全性

在本部分中，我们将基于判定性 MLWR 问题和有偏 MLWR 问题，通过以下定理证明我们 PKE 方案的 IND-CPA 安全性：

4.1.1 IND-CPA 安全性

定理 2. 设 $p = (q - 1)/2^d$, $p' = (q - 1)/2^{d_1}$ 。对任意敌手 $\mathcal{A}$, 存在运行时间与 $\mathcal{A}$ 大致相同的三个敌手 $\mathcal{B}_0, \mathcal{B}_1, \mathcal{B}_2$, 使得

$$Adv_{COMPASS.PKE}^{ind-cpa}(\mathcal{A}) \leq Adv_{\mathcal{H}()}^{prf}(\mathcal{B}_0) + Adv_{n,q,p,k,l}^{mlwr}(\mathcal{B}_1) + Adv_{n,q,p',d,k,l}^{biased-mlwr}(\mathcal{B}_2).$$

证明. 由于我们在所有参数中设 $d_2 \geq d_1$, 可以假设敌手获得 $\bar{c}_2 = \text{Comp}_{d_1}(c'_2)$ 而非 c_2 , 这只会增加敌手的优势。

我们 PKE 方案的 IND-CPA 安全性可以通过一系列安全游戏来证明, 这些游戏在 PRF 安全性、MLWR 的困难性和有偏 MLWR 的困难性下是不可区分的。第一个游戏是真实的 IND-CPA 游戏, 最后一个游戏满足公开参数均匀随机, 且密文通过对均匀随机元素应用压缩函数生成, 因此 IND-CPA 敌手的优势为 0。

- **Game 0:** 这是真实的 IND-CPA 游戏, 公钥和密文由 MLWR-PKE.KeyGen 和 MLWR-PKE.Encrypt 算法生成。
- **Game 1:** 在此游戏中, 我们改变了公钥的生成方式。具体来说, 公钥矩阵 $\hat{\mathbf{A}}$ 是真正均匀随机的, 而非使用 PRF 生成。
- **Game 2:** 在此游戏中, 我们进一步改变了公钥的生成方式。特别地, 公钥向量 $\mathbf{t}'$ 是均匀随机选择的, 而非由算法 MLWR-PKE.KeyGen 生成。
- **Game 3:** 在此游戏中, 我们改变了密文的生成方式, 其中元素 $\mathbf{c}'_1$ 和 c'_2 是均匀随机选择的。

在 PRF 安全性下, **Game 0** 和 **Game 1** 不可区分。

Game 1 和 **Game 2** 由 MLWR 的困难性保证不可区分。

对于 **Game 2** 和 **Game 3**, 设 $\mathbf{A}' = [\mathbf{A}|\mathbf{t}']$, 则 **Game 2** 中元素 $\mathbf{c}_1$ 和 $\bar{c}_2 = \text{Comp}_{d_1}(c'_2)$ 的拼接与 $\mathbf{b} = \lfloor \mathbf{r}^T \mathbf{A}' + \mathbf{0}^T \rfloor \lfloor q/2 \rfloor \cdot m \rfloor_{p'}$ 具有相同分布, 且由有偏 MLWR, 它与 $\lfloor \mathbf{u} + \mathbf{0}^T \rfloor \lfloor q/2 \rfloor \cdot m \rfloor_{p'}$ 独立, 其中 $\mathbf{u}$ 均匀随机, 这等于 **Game 3** 中的分布。因此在有偏 MLWR 假设下, **Game 2** 和 **Game 3** 不可区分。□

该 IND-CPA 安全加密方案是通过适当变换获得 IND-CCA 安全 KEM $\text{KEM} = (\text{Encaps}, \text{Decaps})$ 的基础。最近, Fujisaki-Okamoto 变换的若干后量子版本 [HHK17, JZC⁺18] 及其相应的安全归约已经发展起来。在此, [HHK17] 中的 FO^ℓ 变换配合 Jiang 等人 [JZC⁺18] 的后量子归约, 为非完美正确性方案给出了最紧致的归约。然而, 也可以使用其他变换将 PKE 转化为 CCA 安全的 KEM。

4.1.2 经典随机预言机下的 IND-CCA 安全性

通过将哈希函数 $\mathcal{G}$ 和 $\mathcal{H}$ 建模为随机预言机，可以证明 CCA 安全性的下界。我们使用 Hofheinz 等人 [HHK17] 的安全界，该工作考虑了 Fujisaki-Okamoto 变换的一个 KEM 变体，也可以处理加密方案的小失败概率 δ 。该失败概率必须在密码学上可忽略不计，安全性才能成立。

根据已有工作如 [JZC⁺18]，我们得到随机预言机模型下 KEM 安全性和正确性的以下定理：

定理 3. 在随机预言机模型中，假设 $\mathcal{G}$ 和 $\mathcal{H}$ 为随机预言机，对 IND-CCA 敌手 $\mathcal{B}$ 最多分别向随机预言机 $\mathcal{G}$ 和 $\mathcal{H}$ 进行 $q_{\mathcal{H}}$ 和 $q_{\mathcal{G}}$ 次查询，并向解密预言机进行 q_D 次查询，存在运行时间与敌手 $\mathcal{B}$ 大致相同的 IND-CPA 敌手 $\mathcal{A}$ ，使得：

$$Adv_{COMPASS.KEM}^{ind-cca}(\mathcal{B}) \leq 3Adv_{COMPASS.PKE}^{ind-cpa}(\mathcal{A}) + q_{\mathcal{G}}\delta + \frac{2q_{\mathcal{G}} + q_{\mathcal{H}} + 1}{2^n}.$$

4.1.3 量子随机预言机下的 IND-CCA 安全性

我们可以将 KEM 的 IND-CCA 安全性从底层公钥加密的 IND-CPA 安全性归约出来。

定理 4. 在量子随机预言机模型中，假设 $\mathcal{G}$ 和 $\mathcal{H}$ 为随机预言机，对任意 IND-CCA 量子敌手 $\mathcal{B}$ ，最多分别向量子随机预言机 $\mathcal{G}$ 和 $\mathcal{H}$ 进行 $q_{\mathcal{H}}$ 和 $q_{\mathcal{G}}$ 次查询，并向解密预言机进行 q_D 次（经典）查询，存在运行时间与 $\mathcal{B}$ 大致相同的敌手 $\mathcal{A}$ ，使得：

$$Adv_{COMPASS.KEM}^{ind-cca}(\mathcal{B}) \leq 2q_{\mathcal{H}}\frac{1}{\sqrt{2^n}} + 4q_{\mathcal{G}}\sqrt{\delta} + 2(q_{\mathcal{G}} + q_{\mathcal{H}})\sqrt{Adv_{COMPASS.PKE}^{ind-cpa}(\mathcal{A}) + 1/|M|}.$$

在所有攻击场景中，我们假设量子计算深度限制为 2^{64} 个量子门。

4.2 具体安全性

任何 MLWR 实例都可以转换为规模为 $n \cdot k \times n \cdot \ell$ 的 LWR 实例，即秘密向量长度为 $n_{LWR} = n \cdot \ell$ 且 $m_{LWR} = n \cdot k$ 个样本。在以下讨论中，我们重载符号，令 $(\mathbf{A}, \mathbf{t})$ 代表 LWR 实例而非 MLWR 实例。

LWR 可视为 LWE 的一种变体。具体来说，我们可以将 $\lfloor \frac{p}{q}\mathbf{A}\mathbf{s} \rfloor = \mathbf{t}$ 写成等价形式： $\mathbf{A}\mathbf{s} + \mathbf{e} = \bar{\mathbf{t}}$ ，其中 $\bar{\mathbf{t}} = \frac{q}{p}\mathbf{t} + \frac{q}{2p}\mathbf{1}$ （这里 $\frac{q}{2p}\mathbf{1}$ ，一个所有元素均为 $\frac{q}{2p}$ 的向量，用于将 $\mathbf{e}$ 的均值重置为零，因为我们使用 $\lfloor \cdot \rfloor$ 而非 $\lfloor \cdot \rfloor$ ），且 $\mathbf{e} = \mathbf{t} - \mathbf{A}\mathbf{s}$ 称为舍入误差，可视为 LWE 中的误差向量。这就是我们可以直接将 LWE 的安全评估用于 LWR 的原因。

4.2.1 原始攻击 (Primal Attack)

原始攻击 [AGVW17] 使用 Kannan 嵌入技术 [Kan83] 将 LWE 问题归约为唯一最短向量问题 (uSVP)，然后利用格约简算法恢复目标向量。Albrecht 等人 [AFG13] 首先提

出了针对 LWE 问题的原始攻击。该方法也可以自然地扩展到 LWR，将舍入误差视为待恢复的噪声分量。

对于针对密钥恢复的安全性，攻击可以通过在给定 LWR 样本 $(\mathbf{A}, \mathbf{t})$ 的条件下寻找 LWR 秘密来执行，令 $\tilde{\mathbf{t}} = \begin{pmatrix} \mathbf{t} \\ 0 \end{pmatrix}$, $\bar{\mathbf{t}}$ 如上定义。嵌入格的基构造如下：

$$\mathbf{B} = \begin{pmatrix} \mathbf{A}' & \tilde{\mathbf{t}} \\ \mathbf{0}^T & 1 \end{pmatrix},$$

其中 $\mathbf{A}' = \begin{pmatrix} q\mathbf{I}_{m-n} & \mathbf{A} \\ \mathbf{0} & \mathbf{I}_n \end{pmatrix}$ 。在该嵌入格 $\mathbf{B}$ 中，存在唯一的格最短向量 $\mathbf{v} = (\mathbf{e}, \mathbf{s}, 1)$,

其中 $\mathbf{e}$ 是 LWR 舍入误差。其平方范数约为 $\|\mathbf{v}\|^2 \approx \sigma^2 \cdot m + \frac{q^2}{12p^2} \cdot n$ ，短于格第二短向量 $\lambda_2(\mathcal{L})$ 。因此，LWR 问题归约为嵌入格上的 uSVP 问题。评估嵌入格上 uSVP 的安全评估模型包括：Core-SVP 模型 [ADPS16]、格估算器中的 uSVP 模型以及两步模式安全评估模型 [XWW⁺24]。

对于针对明文恢复的安全性，(有偏) LWR 样本为： $(\mathbf{A} \parallel \mathbf{t}', \mathbf{c}_1 \parallel \mathbf{c}_2)$ 。可以类似地构建格基，这可视作具有更多样本 $(n \cdot (l+1) > n \cdot k)$ ，因为我们对所有安全级别设 $k = l$ 的 LWR 实例。然而，我们声明求解 LWR 的最优样本数低于 nk ，因此增加更多样本对攻破方案安全性没有帮助。所以在以下讨论中，我们仅考虑针对密钥恢复的安全级别。

两步模式下的 LWE 安全评估模型 [XWW⁺24] 为 LWE 困难性分析提供了精化估计，也可应用于上述 LWR 原始攻击。研究表明，与独立的 BKZ 模式 [SE94, CN11] 相比，两步模式可以更高效地求解 uSVP，从而优化 LWR 安全性的评估。下表给出了在经典和量子参数设置下使用精化两步估算器的具体困难性级别。

表 4: 两步模式下的具体安全参数（经典和量子参数设置）[XWW⁺24]

设置	安全级别	n_{LWR}	m_{LWR}	q	p	η_s	Memory($\log_2(\text{bit})$)	Time($\log_2(\text{gate})$)	d_{primal}	β_{BKZ}	β_{sieve}
经典	128	512	512	3329	832	3	99.1	142.64	1025	398	459
经典	256	1024	1024	3329	832	2	199.83	284.1	1912	902	969
经典	384	1536	1536	7681	1920	3	288.77	409.94	2892	1348	1419
经典	512	2048	2048	7681	1920	4	400.26	568.28	3849	1910	1982
量子	128	512	512	3329	832	3	99.82	123.62	1025	394	462
量子	256	1024	1024	3329	832	2	200.56	245.92	1912	897	972
量子	384	1536	1536	7681	1920	3	289.58	354.87	2892	1343	1423
量子	512	2048	2048	7681	1920	4	401.17	491.87	3849	1904	1986

β_{BKZ} : BKZ 预处理的最大的分块大小 β 。

此外，我们比较了本算法与 ML-KEM 的具体安全性。可以看出，在相似安全级别下，本算法的时间复杂度高于 ML-KEM。

表 5: COMPASS-KEM 与 ML-KEM 在原始攻击下具体安全参数的对比。

安全级别	算法名称	Time($\log_2(\text{gate})$)	Memory($\log_2(\text{bit})$)
128-bit	COMPASS-KEM-128	142.6	99.1
128-bit	ML-KEM-512	141.4	98.1
256-bit	COMPASS-KEM-256	284.1	199.8
256-bit	ML-KEM-1024	276.9	194.6

为验证所提方案的安全性，我们在传统原始攻击路径（如 uSVP 和 BDD ）下对 LWR 实例进行了具体参数评估。实验结果表明，在给定相同安全参数设置的情况下，传统攻击路径的计算开销（**Time**）高于表4中所示的**两步模式**。基于两步模式的评估框架更精确地捕获了格约简过程中的安全瓶颈，确保每个安全级别的比特安全性稳健高于目标阈值。

4.2.2 对偶攻击 (Dual Attack)

最近工作 [CMHST25] 对 MATZOV 式对偶攻击进行了精化，显示在其模型下 ML-KEM 的困难性估计远低于原始攻击。然而，他们的估计依赖许多不稳定的启发式假设，且未知是否适用于其他方案。为完整起见，我们也在其模型下给出了方案的安全评估，并显示在所有安全级别下评估结果均超过所需阈值。

表 6: 最新对偶攻击下的具体安全参数（经典和量子参数设置）

设置	安全级别	n_{LWR}	m_{LWR}	q	p	η_s	Time($\log_2(\text{gate})$)	d_{dual}	n_{enum}	n_{fft}	k_{fft}	β_{BKZ}	β_{sieve}
经典	128	512	512	3329	832	3	139.40	477	5	52	9	383	387
经典	256	1024	1024	3329	832	2	259.44	830	10	137	19	810	791
经典	384	1536	1536	7681	1920	3	383.06	1278	7	188	27	1249	1209
经典	512	2048	2048	7681	1920	4	538.73	1889	6	249	39	1801	1735
量子	128	512	512	3329	832	3	123.58	482	1	45	8	387	397
量子	256	1024	1024	3329	832	2	228.36	853	4	125	17	822	809
量子	384	1536	1536	7681	1920	3	335.88	1294	10	165	23	1262	1228
量子	512	2048	2048	7681	1920	4	472.52	1886	5	224	34	1822	1762

根据表6中给出的评估结果，编码对偶过滤阶段的估计代价显著低于相应的原始攻击估计。然而，这一比较应谨慎解读。具体来说，结果显示我们 384 比特安全级别的参数选择仅有 383.06-比特安全性，低于需求阈值。但我们声明，[CMHST25] 的估计仅允许约 31% 的成功率，这意味着真实的安全性比特至少应比估计结果高出 $\log_2(\frac{1}{0.31}) \approx 1.69$ 比特，因此我们 384 比特安全性的参数仍能满足需求。

此外，编码对偶代价模型假设最终的子 LWE 求解步骤可忽略不计，而在实际的密钥恢复场景中，该步骤可能要求解维度为 $n - n_{\text{enu}}$ 的剩余 LWE 实例。若剩余实例通

过原始攻击求解，其代价不应被忽略，尤其当 n_{enu} 较小时。因此，尽管报告的对偶攻击估计显著较低，该模型可能低估了完整秘密恢复的全部代价。

5. 性能

本节对所提 COMPASS-KEM 的性能特征进行全面评估。评估分为两个主要部分：首先，在第5.1节中给出理论性能分析，讨论实现权衡以及由 MLWR 机制带来的效率增益——最显著的是消除了显式噪声采样。其次，在第5.2节中，报告从 ANSI C 参考实现获得的定量基准测试结果。通过评估所有指定安全级别的周期计数，我们旨在证明 COMPASS-KEM 在计算效率和密码学鲁棒性之间实现了优越的平衡，适用于广泛的实用场景。

5.1 性能分析

在本节中，我们考虑 COMPASS-KEM 的实现层面并讨论各种设计权衡。需要指出，当前的性能评估完全基于我们的 ANSI C 参考实现。虽然平台特定优化（如 SIMD 向量化或硬件加速）留待未来工作，但 COMPASS-KEM 的基础设计即使在严格的基础软件环境中也能确保高效率。

5.1.1 实现考量与权衡

NTT 的实现。在实现数论变换时存在许多不同的权衡，主要在代码规模和执行速度之间取得平衡。我们的 COMPASS-KEM C 参考实现利用专用的前向 NTT 和逆向 INTT 算法，利用 ζ 幂的预计算表避免重复的实时计算。在内存受限平台上使用 NTT 的一个关键优势是 R_q 中两个元素的乘法可以在原地计算，无需额外临时存储。尽管当前实现依赖于标准顺序 C 代码，但 NTT 结构非常适合未来的向量化。

对称原语。COMPASS-KEM 内部计算最密集的组件是用于哈希和伪随机数生成的对称原语。在没有并行向量指令的纯 C 实现中，顺序哈希可能成为显著的性能瓶颈。然而，COMPASS-KEM 正是在这种场景下获得了相对于基于 MLWE 方案的关键架构优势：通过依赖确定性舍入机制，完全消除了采样显式噪声向量（如对误差项的 CBD 采样）的需要。所需 PRF 调用次数的减少显著降低了对称密码学开销，使得未优化的 C 实现也能保持高度竞争力。

私钥规模与速度之间的权衡。实现者可以在私钥规模和解封装的计算延迟之间选择不同的权衡。若最小化私钥规模至关重要，可以在私钥中省略存储公钥 pk 及其哈希 $H(pk)$ ，而在解封装期间重新计算它们。此外，由于密钥生成期间的所有随机性均从初始种子派生，理论上可以仅存储 32 字节的核心种子，并在解封装期间重新运行整个密钥生成过程，尽管这会显著增加计算时间。

反之，若私钥规模不是严格约束且解封装速度为优先考虑，则可以决定将完全展开的矩阵 $\hat{\mathbf{A}}$ 作为私钥的一部分存储。这避免了在 Fujisaki-Okamoto (FO) 变换的重新封

封装阶段从种子 ρ 重新计算 $\hat{\mathbf{A}}$ 的昂贵开销。我们的参考实现采用了一种平衡的方法：在 NTT 域中存储多项式以及公钥和 $H(pk)$ ，但不缓存展开的矩阵 $\hat{\mathbf{A}}$ 以保持合理的密钥规模。

静态公钥的本地存储格式。对于用户频繁对同一静态公钥进行封装的应用，可以通过本地缓存展开的公钥（包括矩阵 $\hat{\mathbf{A}}$ 和 $H(pk)$ ）进一步加速封装过程。这节省了在每次封装操作中从种子 ρ 展开矩阵 $\hat{\mathbf{A}}$ 的显著代价以及对 pk 进行哈希的代价。

5.1.2 跨安全级别的性能扩展

COMPASS-KEM 的计算开销在四个指定安全级别（从 128 比特到 512 比特安全性）之间呈现确定性的扩展模式。密钥生成、封装和解封装的周期计数随安全参数自然增长，主要由矩阵维度（ $k \times l$ ）和多项式次数（ n ）的扩展驱动。

矩阵维度（ $k \times l$ ）的影响。KeyGen 和 Encaps 算法中计算最密集的操作是使用基于哈希的可扩展输出函数（XOF）生成伪随机矩阵 $\hat{\mathbf{A}} \in R_q^{k \times l}$ 。因为 $\hat{\mathbf{A}}$ 中多项式元素的个数随 $k \times l$ 成比例增长，XOF 挤压周期与维度参数呈二次关系。例如，从级别 1（ $k = l = 2$ ）过渡到级别 2（ $k = l = 4$ ），同时保持次数 $n = 256$ 不变，导致矩阵展开开销大约增加四倍。

多项式次数（ n ）的影响。对于级别 3 和级别 4，多项式次数从 $n = 256$ 翻倍到 $n = 512$ ，模数 q 从 3329 扩展到 7681。这直接影响数论变换（NTT）和逆向 INTT 操作。由于 NTT 的渐近复杂度为 $O(n \log n)$ ，次数翻倍导致每个向量元素的多项式算术开销略多于一倍。此外，更大的模数和更长的多项式长度使数据内存占用翻倍。这对 CPU 缓存子系统（L1/L2 带宽）施加了更高压力，导致性能扩展曲线在大规模矩阵-向量乘法期间由于内存访问延迟增加而变得略微非线性。

MLWR 扩展优势。从扩展趋势中可以观察到一个关键点：在更高安全级别下对称密码学瓶颈的缓解。在传统的基于 MLWE 的 KEM 中，扩展到更高安全级别需要通过 PRF 采样相应更大的误差向量（ $\mathbf{e}_1, \mathbf{e}_2$ ），这会产生显著且不断增长的哈希开销。由于我们的 COMPASS-KEM 设计通过确定性舍入从本质上消除了显式噪声采样，误差项的生成需要零次 PRF 调用。因此，即使扩展到最苛刻的级别 4（512 比特量子安全， $k = 4, n = 512$ ），对称性能损失仍然严格限定在 $\hat{\mathbf{A}}$ 的展开和转录哈希上。这使总体周期计数显著更低，扩展曲线相比类似的 MLWE 方案更加平坦。

5.2 参考实现与优化实现

根据密码算法提交指南，我们同时提供高度可移植的参考实现和平台优化实现。

5.2.1 可移植 ANSI C 基线

为评估 COMPASS-KEM 的基线效率，我们首先对纯 C 参考实现进行了基准测试。性能分析在配备 AMD Ryzen 7 8745HS 处理器（4 核）运行 Ubuntu 22.04 的工作

站上进行。源代码使用 **GCC 11.4.0** 以 `-O3` 优化标志编译。为确保周期计数测量的可靠性，禁用了 CPU 动态频率调整，将进程绑定到单核，并对报告的时钟周期取平均值以减轻操作系统级别的噪声。

5.2.2 AVX2 优化实现

为补充我们的可移植基线，我们开发了一个针对配备高级向量扩展 2 (AVX2) 指令集的现代 x86 处理器的高度优化实现。对比性能结果详见表7。我们没有依赖通用优化框架，而是设计了一个针对本方案独特代数结构量身定制的 AVX2 架构。该方法充分利用数据级并行，自主加速对称密码原语和核心多项式算术。

表 7: KEM 算法性能对比 (x86 CPU 上的时钟周期)

算法	安全性 (比特)	KeyGen	Encaps	Decaps
ML-KEM-512	128	134,556	159,329	206,350
COMPASS-KEM-128 (C Ref)	128	121,634	161,229	216,570
COMPASS-KEM-128 (AVX2)	128	32,887	28,080	36,118
ML-KEM-1024	256	329,278	369,079	457,594
COMPASS-KEM-256 (C Ref)	256	317,468	387,720	446,243
COMPASS-KEM-256 (AVX2)	256	53,081	55,933	65,690
COMPASS-KEM-384 (C Ref)	384	572,264	826,889	1,093,750
COMPASS-KEM-384 (AVX2)	384	141,590	137,521	166,334
COMPASS-KEM-512 (C Ref)	512	871,519	1,178,215	1,412,591
COMPASS-KEM-512 (AVX2)	512	198,000	183,184	234,086

向量化对称原语 KeyGen 和 Encaps 过程中显著部分的计算延迟被用于展开公钥矩阵 $\hat{\mathbf{A}}$ 的可扩展输出函数 (XOF) 所消耗。为消除此瓶颈 (不论底层密码标准如何)，我们的 AVX2 实现采用了通用的并行哈希方法论。通过利用 256 位 `__m256i` 寄存器，我们同时评估多个独立的哈希状态。具体的多路并行因子与部署的对称标准的内部字长动态对齐。这种交错执行架构最小化了指令开销，并通过充分饱和现代 x86 处理器的宽执行流水线大幅降低了矩阵生成的延迟。

双参数多项式算术与 NTT AVX2 实现的一个显著工程亮点是其对双模数架构的原生支持。为覆盖全部四个安全级别，COMPASS-KEM 操作两组不同的参数集：级别 1 和级别 2 使用 $q = 3329, n = 256$ ；级别 3 和级别 4 使用 $q = 7681, n = 512$ 。

由于两个模数均可安全地适应 13 比特精度，多项式系数紧凑表示为 16 位有符号整数 (`int16_t`)。因此，单个 256 位 AVX2 寄存器恰好打包 16 个系数，实现了高效的

16× 并行因子。为在不引入分支惩罚的情况下最大化指令吞吐量，我们独立设计了两套独立的高度展开的 AVX2 内联函数代码，分别针对每个模数：

- **针对 $q = 3329, n = 256$ ：**我们构造了一套专用的向量化程序，用于 8 层数论变换 (NTT)。Cooley-Tukey 蝶形运算利用 `_mm256_mullo_epi16` 和 `_mm256_mulhi_epi16` 指令，在向量寄存器内原生执行 16 位模运算。
- **针对 $q = 7681, n = 512$ ：**对于该高安全参数集，我们设计了高级的 16 位 SIMD 内存路由和洗牌指令（如 `_mm256_shuffle_epi32` 和 `_mm256_blend_epi32`），以在 9 层 NTT 中无缝处理更大的多项式数组，有效缓解 L1 缓存未命中惩罚。

延迟约简与层次合并。 AVX2 NTT/INTT 中的一项重大算法优化是延迟约简的应用。我们不在每个蝶形层之后执行模约简，而是利用 16 位有符号整数边界 (32, 767)。通过精心跟踪系数增长，我们在 AVX2 寄存器内安全地将值累积到至多 $4q$ ，跨越至多三个融合的蝶形层。严格的向量化 Barrett 约简仅在即将写回内存之前执行。该策略大幅减少了总指令数和流水线延迟。广泛的随机化测试确认，该优化的 SIMD 算术产生与可移植 C 参考实现逐比特相同的结果。

公钥压缩与数据打包 不同于严重依赖拒绝采样的 Fiat-Shamir 签名方案，我们的 COMPASS-KEM 的效率内在依赖于其确定性舍入机制 (`Compd` 和 `Decompd` 函数) 的性能。AVX2 指令集在加速这些操作中发挥了关键作用。我们使用 16 位向量化加法、减法和算术移位指令（如 `_mm256_srai_epi16`）以严格无分支的方式执行舍入过程。这种自主机制允许算法即时压缩密文和公钥系数，无流水线停顿。此外，序列化所需的比特打包和解包程序从头开始完全向量化，用高速 SIMD 移位、混合和掩码替代了顺序按位循环。

未来硬件加速展望 虽然当前 AVX2 实现展示了出色的软件效率，但 COMPASS-KEM 的结构化并行性使其天然适应异构架构。未来工作将探索软件-硬件协同设计方法——例如将对称原语和 NTT 操作卸载到现场可编程门阵列 (FPGA) 或利用新兴的 RISC-V 向量 (RVV) 扩展。这将进一步最大化吞吐量并最小化高性能计算约束下的能耗。

6. 特性声明

在本节中，我们对算法的特性做出声明。

- **创新性。**我们的算法提案基于 MLWR 假设和 MLWR 假设的一个变体。关于 MLWR 假设，不同于 NIST 标准化过程中某些提案（如 Saber 和 Round 5），我们所选参数满足 [LW20] 中的归约要求，即存在理论困难性下界。通过提出一个新定义的 MLWR 变体——即有偏 MLWR 假设，我们修复了 SABER [DKRV18] 文档中有缺陷的安全证明。对于这一新问题，我们给出了严格的定义，并特别首

次建立了从标准 MLWE 问题到该问题的归约（尽管归约使用了已知技术）。在这方面，我们在算法的困难性基础上取得了显著的创新。

- 简洁性。本方案在设计上和实现上均满足简洁性。本方案遵循基于格的加密的标准 LP 范式和针对后量子 KEM 的 QFO 变换，两者均已被充分研究。我们仅使用素模数模格上的困难问题，该问题可以归约到标准 MLWE 从而享有最坏情况安全性，且基于格的困难问题的具体安全性也已得到充分研究。在实现上，我们仅使用基本操作，包括 NTT、模乘和哈希函数。
- 灵活性。除底层密码哈希函数外，本方案的实现仅依赖于模整数操作，可在软件和硬件平台上高效实现，并支持高级别的并行性，允许在效率和资源之间进行灵活平衡。
- 兼容性。我们还在 COMPASS 套件中提出了数字签名方案，两个方案均基于模格，具有相同的多项式次数（虽然模数不同）。将 COMPASS-KEM 与 COMPASS-SIG 结合使用，可实现包括认证密钥交换在内的其他功能。此外，本方案具有比 ML-KEM 更短的公钥和密文规模，因此将本方案适配到现有协议可能比 ML-KEM 更容易。
- 可扩展性。已知基于格的方案具有高度功能性，包括基于身份的加密、基于属性的加密、内积加密和全同态加密。本方案可作为这些更高级方案的构造基础。
- 跨平台实现性能优势。本方案的基本操作单元是 $R_q/(X^n+1)$ 中的多项式 ($n = 256$ 或 $n = 512$)，与基于 RLWE 或 NTRU 的方案相比规模相对较小。虽然 NTT 和模乘操作均可轻松并行实现，但在 256 维或 512 维向量上的 NTT 操作对于资源受限情况也是可接受的。

参考文献

- [ADPS16] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key {Exchange—A} new hope. In *25th USENIX security symposium (USENIX Security 16)*, pages 327–343, 2016.
- [AFG13] Martin R. Albrecht, Robert Fitzpatrick, and Florian Goepfert. On the efficacy of solving LWE by reduction to unique-SVP. Cryptology ePrint Archive, Paper 2013/602, 2013.
- [AGVW17] Martin R. Albrecht, Florian Göpfert, Fernando Virdia, and Thomas Wunderer. Revisiting the expected cost of solving usvp and applications to lwe. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology*

- *ASIACRYPT 2017*, pages 297–322, Cham, 2017. Springer International Publishing.
- [CMHST25] Kevin Carrier, Charles Meyer-Hilfiger, Yixin Shen, and Jean-Pierre Tillich. Assessing the impact of a variant of matzov’s dual attack on kyber. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025*, pages 444–476, Cham, 2025. Springer Nature Switzerland.
- [CN11] Yuanmi Chen and Phong Q Nguyen. Bkz 2.0: Better lattice security estimates. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 1–20. Springer, 2011.
- [DKRV18] Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. Saber: Module-lwr based key exchange, cpa-secure encryption and cca-secure KEM. In Antoine Joux, Abderrahmane Nitaj, and Tajjeeddine Rachidi, editors, *Progress in Cryptology - AFRICACRYPT 2018 - 10th International Conference on Cryptology in Africa, Marrakesh, Morocco, May 7-9, 2018, Proceedings*, Lecture Notes in Computer Science, pages 282–305. Springer, 2018.
- [HHK17] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the fujisaki-okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *Theory of Cryptography - 15th International Conference, TCC 2017, Baltimore, MD, USA, November 12-15, 2017, Proceedings, Part I*, Lecture Notes in Computer Science, pages 341–371. Springer, 2017.
- [JZC⁺18] Haodong Jiang, Zhenfeng Zhang, Long Chen, Hong Wang, and Zhi Ma. Ind-cca-secure key encapsulation mechanism in the quantum random oracle model, revisited. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2018, Proceedings, Part III*, Lecture Notes in Computer Science, pages 96–125. Springer, 2018.
- [Kan83] Ravi Kannan. Improved algorithms for integer programming and related lattice problems. In *Proceedings of the Fifteenth Annual ACM Symposium on Theory of Computing*, STOC ’83, page 193–206, New York, NY, USA, 1983. Association for Computing Machinery.
- [LP11] Richard Lindner and Chris Peikert. Better key sizes (and attacks) for lwe-based encryption. In Aggelos Kiayias, editor, *Topics in Cryptology - CT-RSA 2011 - The Cryptographers’ Track at the RSA Conference 2011, San*

- Francisco, CA, USA, February 14-18, 2011. Proceedings*, volume 6558 of *Lecture Notes in Computer Science*, pages 319–339. Springer, 2011.
- [LW20] Feng-Hao Liu and Zhedong Wang. Rounding in the rings. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part II*, Lecture Notes in Computer Science, pages 296–326. Springer, 2020.
- [SE94] Claus-Peter Schnorr and Martin Euchner. Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical programming*, 66(1):181–199, 1994.
- [XWG24] Wenwen Xia, Geng Wang, and Dawu Gu. Post-quantum backdoor for kyberkem. In Maria Eichlseder and Sébastien Gambs, editors, *Selected Areas in Cryptography - SAC 2024 - 31st International Conference, Montreal, QC, Canada, August 28-30, 2024, Revised Selected Papers, Part I*, Lecture Notes in Computer Science, pages 237–255. Springer, 2024.
- [XWW⁺24] Wenwen Xia, Leizhang Wang, Geng Wang, Dawu Gu, and Baocang Wang. A refined hardness estimation of lwe in two-step mode. In *IACR International Conference on Public-Key Cryptography*, pages 3–35. Springer, 2024.
- [YXP20] Zhaomin Yang, Tianyuan Xie, and Yanbin Pan. Lattice klepto revisited. In Hung-Min Sun, Shiuh-Pyng Shieh, Guofei Gu, and Giuseppe Ateniese, editors, *ASIA CCS '20: The 15th ACM Asia Conference on Computer and Communications Security, Taipei, Taiwan, October 5-9, 2020*, pages 867–873. ACM, 2020.